PROJECT STATUS REPORT

ClaudeCode Advancements

Research, tools, and systems for AI-assisted development

9,393	167,895	9	982
TESTS PASSING	LINES OF CODE	MODULES	COMMITS

40	412	18	34d
master tasks	intelligence findings	live hooks	project age

Session 166 · 2026-03-25

github.com/mpshields96/ClaudeCodeAdvancements

CONTENTS

Contents

1 Executive Summary 3
Project overview, health dashboard, and key metrics

2 Five Frontiers 4
Status of the five core research areas

3 Module Deep-Dives 5
Detailed breakdown of each module's capabilities and progress

4 Master Tasks 9
All aspirational goals — complete, active, pending, and blocked

5 Priority Queue 17
Decay-based priority scoring for active work

6 Live Infrastructure 18
Hook architecture integrated into Claude Code

7 Intelligence & Research 18
Reddit findings, academic papers, and self-learning metrics

8 Architecture Decisions 23
Key technical decisions and their rationale

9 Risks & Blockers 24
Known issues, mitigations, and technical debt

10 Next Priorities 24
Upcoming work items ranked by impact

11 Honest Assessment 25
Objective gaps, limitations, and areas falling short

EXECUTIVE SUMMARY

Executive Summary

ClaudeCodeAdvancements is a research and development project building validated tools, hooks, and systems for Claude Code users and AI-assisted development workflows. Launched 2026-02-19 with 166 development sessions completed. All five original research frontiers are production-complete with 9,393 automated tests across 9 modules. The project has expanded into 40 master-level tasks covering autonomous intelligence gathering, self-learning loops, academic research integration, and professional design output. Zero external Python dependencies.

SINCE LAST REPORT — Session 159 → 166

+434	+8808	+46	+1	+14
Tests	LOC	Commits	Completed MTs	Findings

Newly Completed: MT-22 — Autonomous 1-Hour Loop

Project Health

TESTS

9,393 / 9,393

240 suites — 100% pass rate

CODEBASE

167,895

Source: 68,494 Test: 99,401

Test-to-source ratio: 1.45:1

VELOCITY

982 commits

28.9/day · 5.9/session

34 days · 166 sessions

MODULES

9

169 source files · 241 test files

0 stubs · 0 syntax errors

MASTER TASKS

40

9 done · 25 active · 5 pending

INTELLIGENCE

412 findings

1242 papers · 18 live hooks

Zero external dependencies

FIVE FRONTIERS

Five Frontiers

The five core research areas that drive CCA's development, each validated through community intelligence, Anthropic research, and developer surveys.

1 Persistent Cross-Session Memory

Active CRITICAL

Every session starts from zero — this gives Claude a brain that persists.

TESTS: **340** LOC: **1,758**

2 Spec-Driven Development

Active HIGH

Unstructured prompting produces poor architecture. Enforce requirements > design > tasks > implement.

TESTS: **205** LOC: **1,191**

3 Context Health Monitor

Active HIGH

Context rot causes silent output degradation. Monitor, warn, and auto-handoff.

TESTS: **434** LOC: **2,871**

4 Multi-Agent Conflict Guard

Active HIGH

Parallel agents overwrite each other. Guard credentials, paths, and system modifications.

TESTS: **1073** LOC: **6,196**

5 Usage Transparency Dashboard

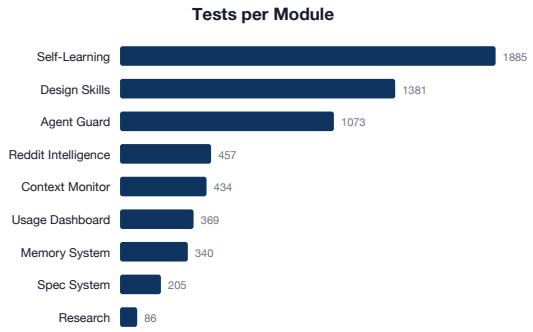
Active MEDIUM

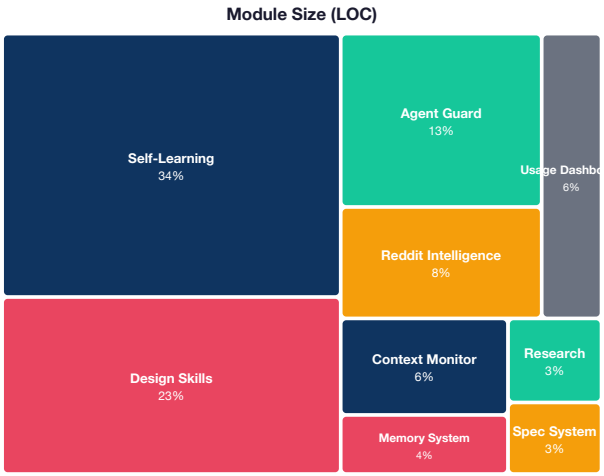
No real-time token/cost visibility. Counter, alerts, and structural completeness.

TESTS: **369** LOC: **3,030**

MODULE DEEP-DIVES

Module Deep-Dives





Memory System

Active

TESTS **340** LOC **1,758** FILES **4** `memory-system/`

Persistent cross-session memory. Captures decisions, preferences, and context across Claude Code sessions using hooks and MCP.

KEY COMPONENTS

- Capture hook (PostToolUse + Stop)
- CLI viewer (stats, search, list)
- 8-char hex ID schema
- MCP retrieval server
- Compaction-resistant handoff

WHY THIS MATTERS Means Claude remembers your preferences and past decisions across sessions — no more re-explaining context every time you start a new chat.

Spec System

Active

TESTS **205** LOC **1,191** FILES **4** `spec-system/`

Spec-driven development workflow. Enforces requirements > design > tasks > implement sequence with approval gates.

KEY COMPONENTS

- /spec:requirements interview scaffold
- /spec:tasks atomic decomposer
- PreToolUse spec validator hook
- /spec:design architecture generator
- /spec:design-review (4 expert personas)
- UserPromptSubmit auto-activation

WHY THIS MATTERS Forces Claude to plan before coding. You get a reviewable blueprint (requirements, design, tasks) instead of spaghetti code from vague prompts.

Context Monitor

Active

TESTS **434** LOC **2,871** FILES **10** `context-monitor/`

Context health monitoring with adaptive thresholds. Tracks token usage, warns at quality ceilings, blocks exit at critical levels.

KEY COMPONENTS

- PostToolUse token meter
- Stop hook auto-handoff
- Automatic session wrap trigger
- PreToolUse threshold alert
- Compact anchor (every 10 turns)
- Session pacer (2-3h autonomous runs)

WHY THIS MATTERS Warns you when Claude's memory is getting full so it doesn't silently start forgetting instructions mid-session. Prevents the 'why did it ignore my rules?' problem.

Agent Guard

Active

TESTS **1073** LOC **6,196** FILES **21** `agent-guard/`

Multi-agent safety, conflict prevention, and Senior Dev code review. Guards credentials, network ports, dangerous paths, and provides automated quality scoring on every write.

KEY COMPONENTS

- Mobile approver (ntfy.sh push)
- Network/port exposure guard
- Path validator (LIVE in hooks)
- File ownership manifest
- Bash command safety guard (LIVE in hooks)
- PR effort scorer (1-5 scale)
- False positive filter
- Tech debt tracker (trend analysis)
- Senior Dev orchestrator hook (LIVE)
- Credential extraction guard
- Content scanner (9 threat categories)
- Session guard (slop detection)
- Edit retry guard (LIVE in hooks)
- SATD detector (TODO/FIXME/HACK markers)
- Code quality scorer (0-100, A-F)
- Review classifier (6 categories)
- ADR reader (architecture decision discovery)

WHY THIS MATTERS Stops Claude from accidentally deleting files, exposing API keys, or breaking your computer when running autonomously overnight.

Usage Dashboard

Active

TESTS **369** LOC **3,030** FILES **7** `usage-dashboard/`

Token and cost transparency. CLI counter, OpenTelemetry receiver, cost alerts, and structural completeness checker.

KEY COMPONENTS

- Token counter CLI
- PreToolUse cost alert hook
- OTLP HTTP/JSON receiver
- /arewedone completeness checker

WHY THIS MATTERS Shows you exactly how many tokens and dollars each session costs, so you can spot waste and stay within your Max subscription limits.

Reddit Intelligence

Complete

TESTS **457** LOC **3,983** FILES **7** `reddit-intelligence/`

Community signal research. Discovers tools, patterns, and pain points from Reddit and GitHub to drive development priorities.

KEY COMPONENTS

- Reddit reader (posts + all comments)
- GitHub repo evaluator
- Subreddit profile registry (10 profiles)
- Autonomous scan pipeline
- Sandboxed repo tester

WHY THIS MATTERS Automatically finds the best new tools, tips, and techniques from Reddit so you don't have to scroll through hundreds of posts yourself.

Self-Learning

Complete

TESTS **1885** LOC **15,860** FILES **39** `self-learning/`

Cross-session improvement through structured observation, pattern detection, autonomous proposals, and research ROI tracking.

KEY COMPONENTS

- JSONL event journal
- Trace analyzer (4 detectors)
- Sentinel adaptive mutation
- Findings resurfacer
- Overnight detector (time-stratified analysis)
- Trade reflector (5 pattern detectors)
- Pattern reflector + strategy engine
- YoYo improvement loop
- Academic paper scanner
- Skillbook injection hook (LIVE)
- Research outcomes ROI tracker

WHY THIS MATTERS Makes your Kalshi bot smarter over time by analyzing what worked and what didn't. The bot learns from its own wins and losses instead of repeating mistakes.

Design Skills

Active

TESTS **1381** LOC **10,662** FILES **17** `design-skills/`

Professional visual output. Typst-based PDF reports, slides, HTML dashboards, SVG charts, website generation, and daily project snapshots.

KEY COMPONENTS

- Report generator (Typst pipeline)
- Dashboard generator (HTML)
- Website generator (landing + docs)
- Design guide + visual language
- Slide generator (16:9 PDF)
- Chart generator (SVG)
- Daily snapshot engine (metric diffs)

WHY THIS MATTERS Generates professional PDF reports and charts automatically — no manual formatting. One command gives you a polished status overview you could show anyone.

Research

Complete

TESTS **86** LOC **1,397** FILES **3** `research/`

R&D tools and experimental capabilities including iOS project generation and Xcode build wrapper.

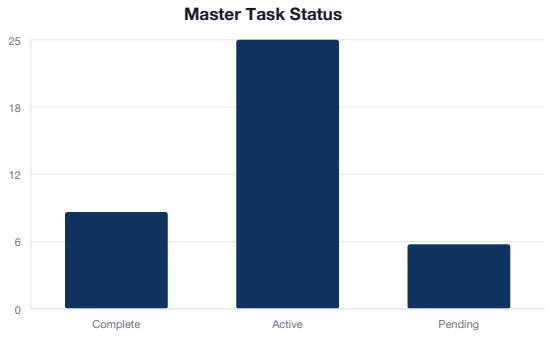
KEY COMPONENTS

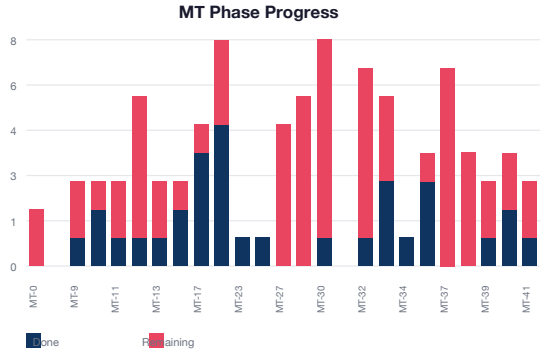
- iOS project generator (SwiftUI)
- Xcode build wrapper

WHY THIS MATTERS R&D sandbox for experimental features. Things get prototyped here before becoming real modules — keeps the main codebase clean.

MASTER TASKS

Master Tasks





40 tasks defined · **9 complete** · **25 active** · 5 pending · 1 blocked

Completed

9 tasks delivered

MT–2 Mermaid Architecture Diagrams in Spec System

Complete

Auto-generates architecture diagrams in the spec workflow. See the design visually before writing code.

MT–3 Virtual Design Team Plugin

Complete

Multiple AI personas (backend, frontend, security) review designs from different expert perspectives.

MT–4 Frontend Design Excellence via Design Vocabulary

Complete

Gives Claude a professional design vocabulary so UI recommendations use real design principles, not vague suggestions.

MT–6 On-Demand Subreddit Scanner

✓ 43 tests Complete

Scan any subreddit on demand for Claude Code relevant posts. The 'nuclear' option for deep intelligence sweeps.

profiles.py: 10 builtin subreddit profiles (4 domains: claude/trading/dev/res... · --profile and --quick flags added to nuclear_fetcher.py

MT–7 Programmatic Trace Analysis for Self-Learning

✓ 50 tests Complete

Finds patterns in how Claude wastes tokens (retries, unnecessary reads) so sessions run faster and cheaper.

self-learning/trace_analyzer.py: TranscriptEntry, TranscriptSession, RetryDet...

MT-20 **Senior Developer Agent** ✓ 890 tests Complete

An automated code reviewer that catches bugs, tech debt, and design issues before they become problems. Like having a senior engineer on call 24/7.

MT-22 **Autonomous 1-Hour Loop** ✓ 66 tests Complete

Makes the whole system run without you. Sessions start, do work, wrap up, and hand off to the next session automatically.

MT-26 **Financial Intelligence Engine** ✓ 43 tests Complete

Financial intelligence tools for the Kalshi bot: regime detection, calibration analysis, order flow. Directly improves bet quality.

MT-29 **Cowork + Claude Pro Bridge Hivemind** Complete

Bridge between Claude Pro web Cowork mode and Claude Code CLI for hivemind coordination across interfaces.

Active

25 tasks in progress

MT-0 **Kalshi Bot Self-Learning Integration** ✓ 24 tests In Progress

Phase 0/2

The single most important task: making the Kalshi bot learn from its own trading results. Every dollar of improvement here compounds over time.

Delivered: Phase 1 (CCA): 6 event types, trading metrics, 4 pattern detectors, 24 tests · Phase 2 (Polybot): Deployed via different implementation path (DB-direct vs J... · kalshi_self_learning.py (916 LOC): Bucket stats, Wilson CI, CUSUM, learning p... · trade_reflector.py (791 LOC): Trade pattern analysis, 5 detectors · calibration_bias.py (426 LOC): Systematic mispricing detection

COMPLETE (Phase 1: S21, Phase 2: deployed via polymarket-bot monitoring chat).

Remaining: Phase 1: S21, Phase 2: deployed via polymarket-bot monitoring chat).

MT-1 **Maestro-Style Visual Grid UI** In Progress

Visual grid dashboard showing all active sessions, system status, and task progress at a glance — like a mission control center.

MOSTLY SELF-RESOLVED (S96). Multiple tools now exist:

MT-9 Autonomous Cross-Subreddit Intelligence Gathering

✓ 54 tests **In Progress**

Phase 1/3

Automates Reddit scanning so high-value posts get flagged without you having to check manually.

Delivered: autonomous_scanner.py: ScanPrioritizer (staleness + yield + diversity scoring... · execute_scan(): End-to-end pipeline — fetch → filter → dedup → classify → rec... · ScanResult dataclass with to_dict/save_json for structured output. · /cca-nuclear autonomous mode: auto-picks highest-priority sub via scanner. · Phase 3: Supervised trial — run with Matthew monitoring for 3 sessions.

Phase 2 COMPLETE (Session 31). Delivered:

Remaining: Phase 3: Supervised trial — run with Matthew monitoring for 3 sessions.

MT-10 YoYo-Pattern Continuous Self-Learning and Self-Building

✓ 64 tests **In Progress**

Phase 2/3

The 'get smarter each session' engine. Tracks what strategies work and surfaces the best ones automatically.

Delivered: improver.py: ImprovementProposal lifecycle, ImprovementStore (JSONL), Proposa... · QualityGate: Geometric mean anti-gaming (Nash 1950 / sentrux pattern). Preven... · Phase 2 COMPLETE (Sessions 42-44): 5/5 validation sessions, 36 transcripts an... · Phase 3: Two tracks: (A) Graduate self-learning to Kalshi bot (requires cross...

Phase 1 COMPLETE (Session 28), QualityGate added (Session 29), E2E validated (Session 29).

Remaining: Phase 3: Two tracks: (A) Graduate self-learning to Kalshi bot (requires cross-project ...

MT-11 Autonomous GitHub Repository Intelligence

✓ 62 tests **In Progress**

Phase 1/3

Scans GitHub trending repos for tools, techniques, and patterns that could improve CCA or the Kalshi bot.

Delivered: github_scanner.py: RepoMetadata (from GitHub API), RepoEvaluator (0-100 scori... · fetch_trending(): GitHub search API with date filters to approximate trending... · _build_trending_query(): configurable language, days, min_stars filters. · TrendingScanner class: per-language scanning, trending history JSONL log, CCA... · Phase 3: Wire trending scanner into MT-9 autonomous pipeline for scheduled runs.

Phase 2 COMPLETE (Session 83). Delivered:

Remaining: Phase 3: Wire trending scanner into MT-9 autonomous pipeline for scheduled runs.

MT-12 Academic Research Paper Integration

✓ 89 tests **In Progress**

Phase 1/6

Finds relevant academic papers (trading math, AI research) and summarizes them so you get the insights without reading 30-page PDFs.

Delivered: self-learning/paper_scanner.py: Semantic Scholar + arXiv API integration, 7 C... · self-learning/paper_digest.py: Kalshi/CCA digest generator with cross-chat br... · Phase 1 (S38): Live API validated, first paper logged · Phase 2: Ran across 4 domains, increased delay to 3s (429 rate limit) · Phase 3 (S100): paper_digest.py built, expanded prediction/statistics queries...

Phase 4 COMPLETE (Session 101). Delivered:

Remaining: Phase 2: Ran across 4 domains, increased delay to 3s (429 rate limit)

MT–13 iOS App Development Capability

✓ 57 tests **In Progress**

Phase 1/3

Generate complete Xcode projects with Swift code and tests from Claude. Prototype iOS apps without manual setup.

Delivered: ios_project_gen.py: Generates complete Xcode projects from CLI (SwiftUI + tes... · xcode_build.py: Python xcodebuild wrapper (build, clean, test, scheme/simulat... · Phase 3: Build first real app (Kalshi mobile dashboard) or install SwiftUI Ag...

Phase 2 COMPLETE (Session 49). Delivered:

Remaining: Phase 3: Build first real app (Kalshi mobile dashboard) or install SwiftUI Agent Skill

MT–14 Autonomous Re-Scanning of Previously Scanned Subreddits

In Progress

Phase 2/3

Re-scans previously visited subreddits to catch new posts and updates since last scan. Keeps intelligence fresh.

Delivered: rescan_sub() method on AutonomousScanner: delta-scanning with timestamp filte... · get_stale_subs() helper for identifying overdue subs · Phase 2: Run rescan on a real stale sub, validate delta filtering works. Phas...

Phase 1 COMPLETE (Session 35). Delivered:

MT–17 UI/Design Excellence and Professional Report Generation

✓ 42 tests **In Progress**

Phase 4/5

One-command professional reports. Instead of explaining project status verbally, just generate a PDF.

Delivered: design-skills/ module with CLAUDE.md, design-guide.md (color palette, typogra... · design-skills/report_generator.py — CCADataCollector + ReportRenderer + CLI (... · design-skills/templates/cca-report.typ — Professional status report template · /cca-report slash command for one-command PDF generation · Phase 2 COMPLETE (Session 47): slide_generator.py + /cca-slides command (25 t...

Phase 1 COMPLETE (Session 41). Delivered:

Remaining: Phase 5: Website templates - landing pages, documentation sites · Phase 5: Website templates — landing pages, documentation sites

MT–21 Hivemind Multi-Chat Coordination

✓ 69 tests **In Progress**

Phase 5/8

Lets multiple Claude sessions work in parallel on different tasks — like having a small dev team instead of one assistant.

Infrastructure COMPLETE. Phase 1 validation IN PROGRESS (S86).

Remaining: Phase 1: Validated 2-chat (Desktop + 1 CLI), 3-5 sessions, must pass all gates · Phase 2: Hardened 2-chat with complex tasks, 3-5 sessions · Phase 3: 3-chat operation (Desktop + 2 CLI), only after Phase 2 gate passes

MT–23 Mobile Remote Control v2

In Progress

Mobile control v2 via Telegram/Discord. Richer interface than push notifications — send commands, receive status updates.

EXTERNALLY RESOLVED (S112). Claude Code Channels shipped March 20, 2026.

Remaining: Install Telegram channel plugin and test with Kalshi bot session.

MT–24 Visualization & Graphics Engine

✓ 41 tests **In Progress**

Phase 1/1

SVG chart engine with 20+ chart types. Powers all data visualization across reports, dashboards, and presentations.

Phase 1 COMPLETE (S111) — `trading_chart.py` (5 SVG chart types: PnL curve, win rate, strategy matrix, drawdown, heatmap). 41 tests.

MT–27 CCA Nuclear v2

✓ 27 tests **In Progress**

Phase 0/5

Smarter Reddit scanning with less noise. Better signal-to-noise means fewer irrelevant posts cluttering your review.

Delivered: Phase 1: APF analysis — "Other" category dropped from 124/335 (37%) to 0/335 ... · Phase 2: classify_post HAY expansion — 28 new sentiment/opinion HAY keywords ... · Phase 3: apf_checkpoint() function + scan report integration. 6 new tests. · Phase 4 (S129/S132): 3-tier NEEDLE precision improvement. Split keywords into... · Phase 5 (S115): apf_session_tracker.py — per-session APF trend tracking. Appe...

Phases 1-5 COMPLETE.

Remaining: Phase 1: APF analysis — "Other" category dropped from 124/335 (37%) to 0/335 (0%). Exp... · Phase 2: classify_post HAY expansion — 28 new sentiment/opinion HAY keywords (vibe cod... · Phase 3: apf_checkpoint() function + scan report integration. 6 new tests.

MT–28 Self-Learning v2

✓ 73 tests **In Progress**

Phase 0/6

Self-learning v2: the bot doesn't just track results, it evolves its own principles about what works. Adaptive strategy, not static rules.

Delivered: Phase 1: principle_registry.py (73 tests) — Laplace-scored principles by domain · Phase 2: pattern_registry.py + detectors.py (42 tests) — Plugin detector arch... · Phase 3: principle_transfer.py (34 tests) — Cross-domain affinity-based transfer · Phase 4: outcome_feedback.py (16 tests) — Research outcomes → principle scoring · Phase 5: predictive_recommender.py (40 tests) — Pre-session ranked recommenda...

COMPLETED (S111). All 6 phases done:

Remaining: Phase 1: principle_registry.py (73 tests) — Laplace-scored principles by domain · Phase 2: pattern_registry.py + detectors.py (42 tests) — Plugin detector architecture · Phase 3: principle_transfer.py (34 tests) — Cross-domain affinity-based transfer · Phase 4: outcome_feedback.py (16 tests) — Research outcomes → principle scoring · Phase 5: predictive_recommender.py (40 tests) — Pre-session ranked recommendations

MT-30 Session Daemon

✓ 116 tests **In Progress**

Phase 1/8

Auto-start and manage all your Claude sessions from one command. No more manually opening 3 terminal windows.

Delivered: Phase 2 (S111): session_registry.py (60 tests) + tmux_manager.py (40 tests) · Phase 3 (S112): session_daemon.py — poll loop, health checking, spawn/restart... · Phase 4 (S113): Integration tests — lifecycle, peak transitions, crash recove... · Phase 5 (S113): Hardening — _mark_exhausted_sessions FAILED state fix, log ro... · Phase 6 (S126): cca_autoloop.py — reads SESSION_RESUME.md, spawns claude, loo...

Phase 8 COMPLETE (S128). S129: +preflight command, +rich --status with audit log, +AUTOLOOP_SETUP.md. Ready for live supervised dry run.

Remaining: Phase 2: session_registry.py + tmux_manager.py (building blocks) · Phase 3: session_daemon.py (poll loop, health checking, spawn/restart) · Phase 4: Integration testing (dry run with 2 sessions) · Phase 5: Hardening (max restart limits, PID file, log rotation)

MT-31 Gemini Pro Integration

In Progress

Use Google's Gemini Pro for visual/design tasks where it excels, while Claude handles the code. Best of both models.

S125 E2E validation: Gemini Flash WORKS via @rlabs-inc/gemini-mcp (v0.8.1). Pro UNAVAILABLE — Google One gives web app Pro access only, not API access. API Pro requires separate Google Cloud billing. Matthew directive: Flash only (Option 1). Scope narrowed to Gemini Flash integration.

MT-32 Visual Excellence & Design Engineering

✓ 890 tests **In Progress**

Phase 1/7

Makes all visual output (reports, charts, dashboards) look professional. Design quality you'd expect from a real product.

NEW (S118). Phase 1 starting — report_charts.py integration + research scan.

Remaining: Phase 2: Advanced chart types (interactive, animated, publication-quality statistical ... · Phase 3: Design system v2 (design tokens, lint rules, cross-format consistency) · Phase 4: UI component library (reusable patterns for Claude Code web outputs) · Phase 5: Dashboard v2 (interactive, real-time, responsive, themeable) · Phase 6: Figure/image generation pipeline (multi-panel, annotation, export)

MT-33 Strategic Intelligence Report

In Progress

Phase 3/6

Combines Kalshi trading data, self-learning intelligence, and research outcomes into one strategic report. Answers: is the bot getting smarter and more profitable?

Unknown

Remaining: Phase 1: Deep dive research — what metrics matter, what data is available, what charts... · Phase 2: Data collection layer — SQL queries, data transformers, schema mapping · Phase 3: Report section templates — Typst layouts for each pillar · Phase 4: Chart integration — wire real data into chart_generator.py chart types · Phase 5: Self-reference hook — JSON export, report diffing, chat-parseable format

MT-34 Medical AI Tool — OpenEvidence Replacement

In Progress

Medical AI research tool for provider-grade clinical decision support. Uses same research rigor as Kalshi bot — academic evidence first, no speculation.

IDEA LOGGED — Matthew needs more time to refine the concept. Do NOT start building until Matthew gives the green light.

MT-35 Background Autoloop — Non-Intrusive Desktop Loop

✓ 23 tests **In Progress**

Phase 3/4

Makes the autoloop unobtrusive — runs in the background, pauses when you're active, resumes when idle. The system works FOR you, not against you.

Delivered: Phase 1 COMPLETE (S142): save/restore frontmost app around autoloop trigger (... · Phase 2 COMPLETE (S143): idle detection via CoreGraphics CGEventSourceSeconds... · Phase 3 COMPLETE (S144): ntfy.sh loop health notifications. notify_loop_healt... · Phase 4: TODO — keyboard shortcut to pause/resume loop

- Phase 1 COMPLETE (S142): save/restore frontmost app around autoloop trigger (~3-5s takeover)

Remaining: Phase 4: Keyboard shortcut to pause/resume loop without killing it · Phase 4: TODO — keyboard shortcut to pause/resume loop

MT-37 AI-Driven Investment Research & Portfolio Intelligence

In Progress

Phase 0/7

Long-term investment research engine — ETF analysis, portfolio construction, factor models. Same rigor as Kalshi: academic basis + backtesting + math validation.

IDEA LOGGED (S148) — Long-term, research-first. Do NOT start building until Phase 1 research is complete. No rush — Matthew directive.

Remaining: Phase 1: Deep academic research survey — produce RESEARCH.md with literature review, m... · Phase 2: Data pipeline — FRED API integration, ETF universe builder, basic factor load... · Phase 3: Portfolio constructor — implement 3-5 allocation strategies with backtest fra... · Phase 4: Risk analysis toolkit — VaR/CVaR, Monte Carlo, stress testing · Phase 5: Tax optimization layer — asset location, TLH simulation

MT-38 Peak/Off-Peak Token Budget System

✓ 21 tests **In Progress**

Phase 0/4

Rate limit awareness — automatically adjusts token usage based on time of day. Heavy work at night, light work during peak hours. Prevents rate limit hits.

Phase 1-3 COMPLETE (S154-S155). Rule active globally. token_budget.py built (21 tests, S154). Peak-hour agent blocking hook active (S155). Phase 4 (autoloop scheduling) remaining.

Remaining: Phase 2: ** Build `token_budget.py` utility that: · Phase 3: ** Hook integration — PreToolUse hook that warns/blocks expensive operations ... · Phase 4: ** Autoloop scheduling — autoloop prefers off-peak windows, defers heavy task...

MT-39 Priority Picker Overhaul — Dust Detection

In Progress

Phase 1/3

Phase 1 COMPLETE (S160). 93 existing tests still pass. `python3 priority_picker.py dust` now shows 15 dusty MTs.

Remaining: Phase 2: Wire dust report into /cca-init briefing — auto-surface dusty MTs at session ... · Phase 3: Auto-archive policy — MTs blocked 100+ sessions with no self-resolution note ...

MT-40 **Automated Nuclear Scanning Loop** ✓ 17 tests **In Progress**

Phase 2/4

Phase 1-2 COMPLETE (S160). Phase 3-4 remaining.

Remaining: Phase 3: Auto-trigger nuclear scan when top_target stale (not yet wired into /cca-auto) · Phase 4: Results auto-feed into MT-41 (synthetic origination)

MT-41 **Synthetic MT Origination** ✓ 22 tests **In Progress**

Phase 1/3

Phase 1 COMPLETE (S160). mt_originator.py built (22 tests). Phase 2-3 remaining.

Remaining: Phase 2: Auto-generate MT proposals with name, description, technical path, and estima... · Phase 3: Surface proposals in /cca-init briefing and priority_picker. Matthew approves...

Pending & Blocked

6 tasks awaiting start or unblocked

MT-5 **Claude Pro ↔ Claude Code Bridge** **Not Started**

Bridges Claude Pro web chat with Claude Code CLI sessions. Share context and continue conversations across interfaces.

MT-8 **iPhone Remote Control Perfection** **Not Started**

Control Claude Code sessions from your iPhone. Approve, reject, or steer work while away from your desk.

MT-18 **Academic Writing Workspace** **Not Started**

Academic writing workspace — structured paper composition with citation management. Professional research output.

MT-19 **Local LLM Fine-Tuning Capability** **Not Started**

Fine-tune local LLMs on CCA data. Long-term goal: specialized models trained on your specific workflows.

MT-25 **Presentation Generator** **Blocked**

Generate presentation slides matching Matthew's preferred style. One command from data to polished deck.

MT-36 **Session Efficiency Optimizer — Quality-Preserving Speed** **Not Started**

Measures and optimizes session overhead (init, wrap, tests) without cutting quality. Every second saved on overhead is a second spent shipping code.

PRIORITY QUEUE

Priority Queue

Active tasks ranked by decay-based priority scoring. Score = base value + aging penalty. Higher = work on this first.

1	MT–21	**Hivemind coordination** Phase 2 PASSED (6th). Phase 3 = 3-chat	10
2	MT–33	**Strategic Intelligence Report** Phase 4-6: chart integration, self-reference, hardening. kalshi_data_collector + learning_data_collector + report_differ built (S122-S123).	10
3	MT–23	**Mobile Remote Control v2** Research: evaluate Telegram/Discord channels MCP	9
4	MT–26	**Financial Intelligence Engine** CCA scope COMPLETE. Tier 3 Phase 2 deferred (needs numpy). 79 pipeline tests.	8
5	MT–32	**Visual Excellence & Design Engineering** Phase 2+: nuclear scan, Kalshi charts, consistency. report_charts wired (S117).	8
—	MT–34	**Medical AI (OpenEvidence replacement)** BLOCKED: Matthew refining concept. Do not start.	6
8	MT–29	**Cowork + Pro Bridge Hivemind** Research: evaluate Cowork, bridge Pro↔Code	5
9	MT–18	**Academic writing** Research: install/evaluate ClaudePrism	7
10	MT–13	**iOS/macOS app dev** Phase 3: first real app	7

LIVE INFRASTRUCTURE

Live Infrastructure

18 hooks integrated into Claude Code via settings.local.json. Every tool call passes through this pipeline.

Event	Matcher	Hook	Purpose
PostToolUse	*	meter.py	Token counter
PostToolUse	*	compact_anchor.py	Context anchor writes
PostToolUse	*	senior_dev_hook.py	SATD + effort + quality scoring
PostToolUse	*	queue_hook.py	Cross-chat queue context
PreToolUse	*	alert.py	Warn/block at red/critical
PreToolUse	*	cost_alert.py	Cost threshold warning
PreToolUse	*	path_validator.py	Dangerous path detection
PreToolUse	*	edit_guard.py	Edit retry prevention
PreToolUse	*	validate.py	Spec guard + plan compliance
PreToolUse	Bash	credential_guard.py	Credential extraction guard
PreToolUse	Bash	bash_guard.py	Command safety (network, packages, system)
UserPromptSubmit	*	skill_activator.py	Spec auto-activation
UserPromptSubmit	*	skillbook_inject.py	Strategy injection
UserPromptSubmit	*	capture_hook.py	Real-time memory capture
UserPromptSubmit	*	queue_hook.py	Cross-chat queue injection
Stop	*	auto_handoff.py	Block exit at critical
Stop	*	capture_hook.py	Session-end memory capture
PostCompact	*	post_compact.py	Recovery + journal logging

INTELLIGENCE & RESEARCH

Intelligence & Research

Reddit Intelligence

FINDINGS BY VERDICT

BUILD	27
ADAPT	61
REFERENCE	267
PERSONAL	0
SKIP	57

7 subreddits · 30 repos evaluated

Self-Learning System

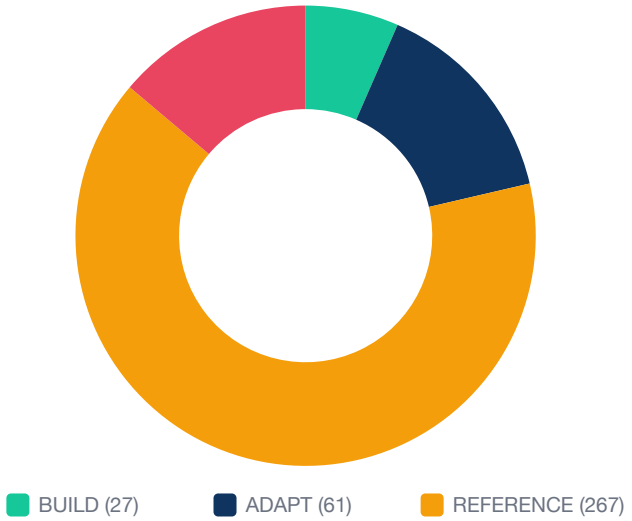
122 PRINCIPLES

79/57 WINS / PAINS

Avg Score	0.5 (higher = more validated)
Journal Sessions	119 tracked
Papers Logged	**1242**
Sentinel Rate	0%

RESEARCH ROI **46** deliveries → **0** implemented → **0** profitable

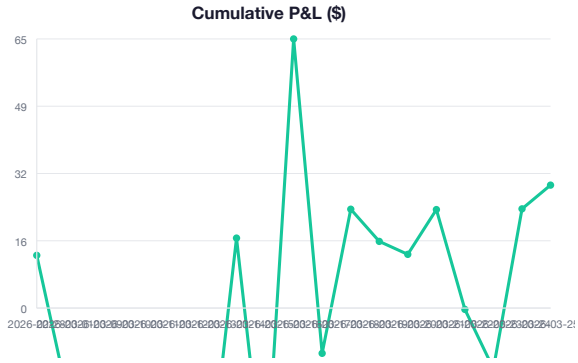
Intelligence Verdicts

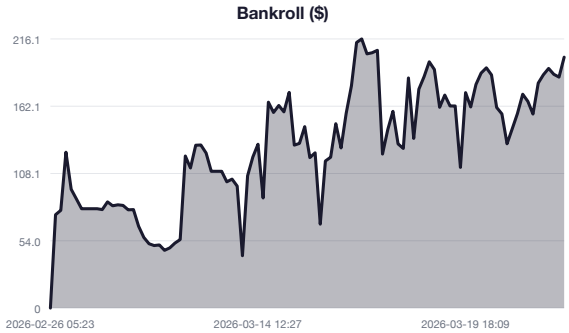


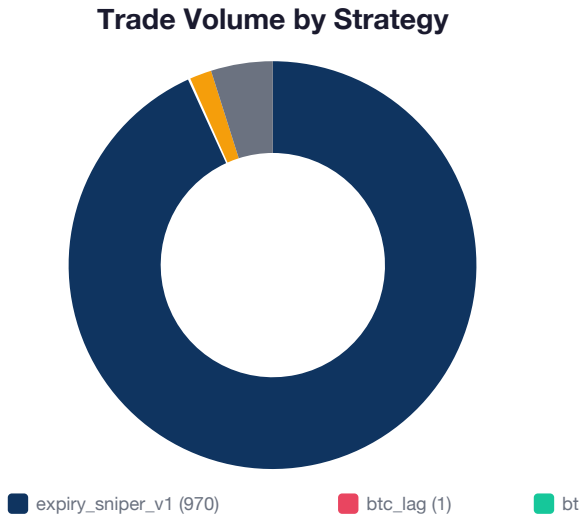
FINANCIAL ANALYTICS — KALSHI BOT

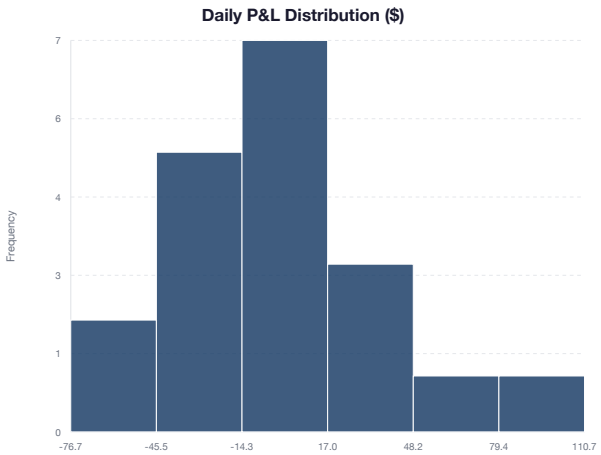
Financial Analytics — Kalshi Bot

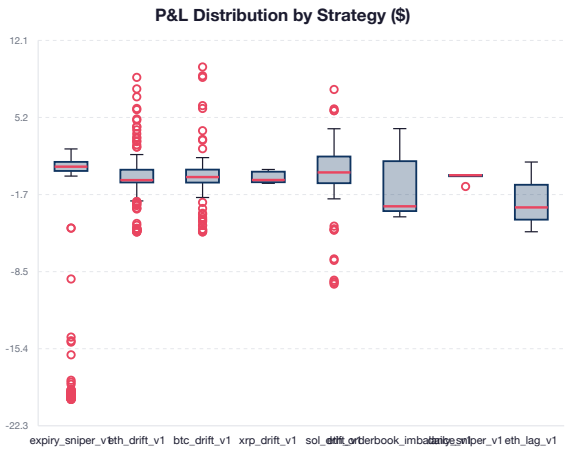
LIVE TRADES	WIN RATE	TOTAL P&L	SETTLED
1342	81.8%	$29.99	1330











Strategy Performance

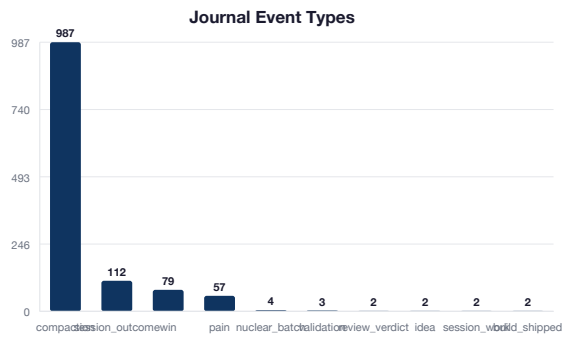
Strategy	Trades	Wins	Win %	Avg P&L	Total P&L
expiry_sniper_v1	970	906	93.6%	$0.11	**$105.8**
btc_lag	1	1	100%	$3.5	**$3.5**
btc_lag_v1	1	1	100%	$0.57	**$0.57**
daily_sniper_v1	18	7	87.5%	$−0.05	**$−0.39**
xrp_drift_v1	51	24	47.1%	$−0.07	**$−3.8**
eth_lag_v1	3	1	33.3%	$−2.18	**$−6.53**
btc_drift_v1	80	40	50%	$−0.12	**$−9.53**
sol_drift_v1	45	30	66.7%	$−0.31	**$−14.08**

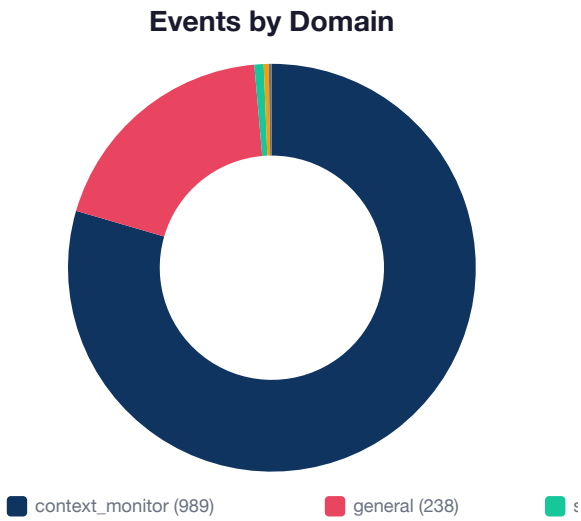
eth_orderbook_imbalance_v1	15	5	33.3%	$−1.21	**$−18.2**
eth_drift_v1	158	73	46.2%	$−0.17	**$−27.35**

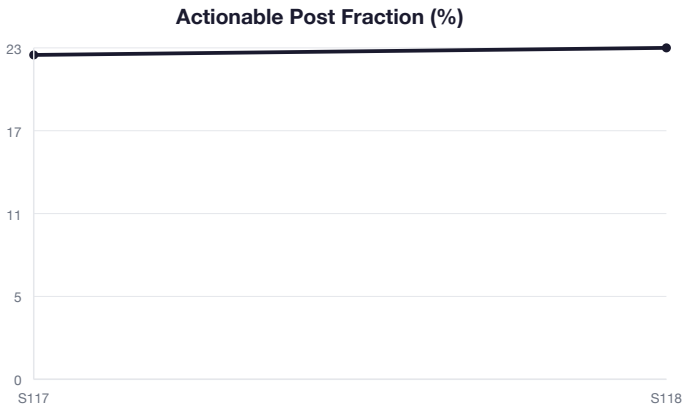
SELF-LEARNING INTELLIGENCE

Self-Learning Intelligence

JOURNAL ENTRIES	WINS / PAINS	CURRENT APF	POSTS REVIEWED
1259	79 / 57	23.2%	340







ARCHITECTURE DECISIONS

Architecture Decisions

Key technical decisions and their rationale. Each decision was made to optimize for simplicity, reliability, or developer experience.

Local-first storage

User owns all data, no cloud dependency

Stdlib-first (no pip)

Zero dependency management overhead

Hook-based delivery

Native Claude Code integration points

Adaptive context thresholds

Fixed percentages break at 1M windows

JSONL for all logging

Append-only, grep-friendly, no corruption risk

QualityGate geometric mean

Prevents Goodhart's Law gaming of metrics

Whitelist-first phishing detection

Zero false positives on legitimate domains

Typst over WeasyPrint

Single binary, millisecond compile, JSON-native

RISKS & BLOCKERS

Risks & Blockers

MT-25 Presentation Generator (Matthew's Style)

BLOCKER

NOT STARTED. BLOCKED on style samples from Matthew.

2 MTs stagnating

RISK

2 active tasks have negative stagnation scores — no progress in recent sessions.

Mitigation: Review priority picker output. Consider deprioritizing or archiving stalled work.

5 severity-3 learnings active

DEBT

High-severity patterns requiring ongoing vigilance across all sessions.

Mitigation: Encoded in global rules. Auto-enforced by hooks where possible.

NEXT PRIORITIES

Next Priorities

1. **MT-32 Phase 7 continued**
   Wire figures into slides, dashboard, website pipelines
2. **economics_sniper_v1 validation (April 8 CPI data)**
3. **MakerSniperStrategy implementation (Kalshi monitoring chat builds from REQ-039 spec)**
4. **Ongoing cross-chat coordination (check POLYBOT_TO_CCA.md each session)**

5 **REQ-035 response**
daily_sniper interim analysis (Wilson CI, sizing recommendation)

HONEST ASSESSMENT

Honest Assessment

Objective gaps, limitations, and areas where the project falls short of its stated goals. Included for accountability.

13 master tasks stalled at Phase 1

GAP

MT-0, MT-9, MT-11, MT-12, MT-13, MT-27, MT-28, MT-30, MT-32, MT-37, MT-38, MT-39, MT-41 — started but no subsequent phases shipped. Risk of scope sprawl without depth.

ACTION: Pick one stalled MT and advance it to Phase 2 this week. Suggested: MT-0 (oldest).

Single-developer project with no external users

LIMITATION

All tooling built for one developer. No community testing, no feedback loop from other Claude Code users. Limits validation of 'community-demanded' features.

ACTION: Consider publishing 1-2 standalone tools (e.g. context monitor, memory system) as open-source to get external feedback.

High test count (6,230) may overstate coverage

NUANCE

Average 56 tests per source file. Many test trivial properties (dict keys, return types). Integration test coverage across modules is limited to 1 suite.

ACTION: Add 3-5 cross-module integration tests that exercise real workflows (e.g. memory capture → retrieval, context alert → handoff).

1 task(s) blocked with no resolution timeline

BLOCKER

MT-25: Presentation Generator

ACTION: Review each blocked MT: either unblock with a workaround, rescope to remove the blocker, or archive if no longer relevant.

CI/CD pipeline exists but unverified

NUANCE

GitHub Actions workflow defined but no evidence of runs in git history. All testing is local-only.

ACTION: Push to GitHub and verify the Actions workflow runs. Fix any failures. Add status badge to README.

STATUS

All Systems Operational

9,393	167,895	166	982
TESTS	LOC	SESSIONS	COMMITS

9 modules · 40 master tasks · 240 test suites · Zero external dependencies

9	25	412
MTs Complete	MTs Active	Findings

Generated 2026-03-25 · Session 166 · github.com/mpshields96/ClaudeCodeAdvancements